

Good question — this is exactly the *right* way to think about a **final-year project**. Your Redis-like server is **infrastructure**, so examiners won't be impressed unless they see **real systems built on top of it**.

Below are **strong, defensible application ideas** that *naturally* demonstrate why your system exists. I'll rank them from **most impactful for FYP** → **simpler but still valid**.

---

## 1. Real-Time Chat System (BEST choice for FYP)

### What you build

- A chat backend where:
  - Messages are stored in your Redis clone
  - Online users are tracked in Sets
  - Message queues use Lists (LPUSH / RPOP)

### How your DB is used

| Feature        | Your Redis Command                           |
|----------------|----------------------------------------------|
| Store messages | LPUSH chat:room1 "hello"                     |
| Fetch messages | LRANGE chat:room1 ( <i>you'll add this</i> ) |
| Online users   | SADD online_users user1                      |
| Presence       | EXISTS user                                  |

### Why examiners like this

- Distributed systems
- Concurrency
- Real-time behavior
- Clear justification for an in-memory DB

**Add-on:** WebSocket server + React UI

**Viva killer line:**

“Traditional SQL databases introduce latency for real-time messaging, so we used an in-memory RESP-based store inspired by Redis.”

---

## 2. Rate Limiter + API Gateway (Very impressive)

### What you build

A mini API gateway that:

- Protects APIs from abuse
- Uses your Redis clone for request counting

### How your DB is used

```
Copy codeINCR user:123:req_count  
EXPIRE user:123:req_count 60
```

(You’ll add INCR, EXPIRE later — that’s good)

### Features

- Per-IP or per-user rate limiting
- Sliding window counters
- Configurable limits

### Why this screams “systems project”

- Performance critical
- Time-based eviction
- Real industry use case

**Bonus:** Compare with Nginx / Redis in performance graph

---

## 3. Session Store for Web Applications

### What you build

- A web app (login, dashboard)
- Sessions stored in **your Redis**, not cookies

## How your DB is used

| Feature       | Usage                    |
|---------------|--------------------------|
| Login session | SET session:uuid user_id |
| Expiry        | TTL support              |
| Fast lookup   | O(1) in-memory           |

## Why this works for FYP

- Clear backend relevance
- Security discussion
- Scales well conceptually

**Great if your panel likes “web + systems” hybrids**

---



## 4. Real-Time Analytics / Counters Dashboard

### What you build

- Live dashboard showing:
  - o Page views
  - o Button clicks
  - o Active users

## How your DB is used

Copy codeINCR page:/home

INCR page:/login

Frontend polls or subscribes → updates charts.

## Why this is safe

- Easy to explain
  - Shows performance advantage clearly
  - Less risky than chat
-

## 5. Distributed Task Queue (Advanced, optional)

### What you build

- Producer pushes jobs
- Workers consume jobs
- Failover handling

### How your DB is used

```
Copy codeLPUSH task_queue job1
RPOP task_queue
```

### Why this is advanced

- Concurrency
- Fault tolerance
- Backpressure concepts

This one impresses **faculty with systems background**.

---

## What I strongly recommend (best combo)

### Main Project

**“Design and Implementation of a Redis-Inspired In-Memory Data Store with Real-Time Applications”**

### Demo Application

-  **Real-time chat OR rate limiter**
  -  Web UI + backend service
  -  Performance comparison vs file/SQL storage
- 

### What NOT to do (important)

-  Just a CLI demo
-  Only showing SET/GET

- ✗ No real application using it
  - ✗ No performance justification
- 

If you want, next we can:

1. Pick **ONE** application and design its architecture
2. Decide **which Redis commands you must implement**
3. Prepare **FYP documentation structure**
4. Design **viva answers** (they *will* ask “why Redis?”)

Tell me which option you want to lock in — I’ll help you make it bulletproof.